

Design - Revision 3

1. Storage Model

```
tasks(  
  
    id TEXT PRIMARY KEY,  
  
    title TEXT,  
  
    owner_session_uuid TEXT,  
  
    notify_target TEXT,  
  
    observable_type TEXT,  
  
    observable_id TEXT,  
  
    checkin_every_minutes INTEGER,  
  
    started_at TEXT,  
  
    last_observed_at TEXT,  
  
    last_observed_status TEXT,  
  
    observation_source TEXT,  
  
    last_notification_sent_at TEXT,
```

```
last_notification_status TEXT,  
  
resolution_status TEXT,  
  
resolution_at TEXT  
  
)  
  
task_events(  
  
    id INTEGER PRIMARY KEY,  
  
    task_id TEXT,  
  
    event_type TEXT,  
  
    observed_status TEXT,  
  
    message TEXT,  
  
    created_at TEXT  
  
)
```

2. Key Design Clarifications

- **last_observed_status** = latest non-terminal observed state from the watchdog.
- **resolution_status** = confirmed terminal outcome used to close the task (e.g. completed, failed).

3. Observable Identity for Process Tasks

Phase 1 shall not use a raw PID as the primary observable identity.

Preferred pattern:

- `observable_type = marker`
- `observable_id = /absolute/path/to/task-marker.json`

The observed process owns and updates the marker file. PID may be stored as secondary evidence but not as the primary identity.

4. Watchdog Logic

1. Load active tasks from SQLite.
2. Inspect observation source.
3. Compute current observed state.
4. Apply unknown threshold (default 2 cycles).
5. Apply notification suppression rules.
6. Emit via `openclaw message send` if due.
7. Write event row and updated task state transactionally.

5. Deterministic Templates

Phase 1 notification messages use templates rather than LLM generation, e.g.:

```
[Agent Follow-Up] Task still running: Projects page refactor.
```

```
Observed status: running.
```

```
Last observed: 2026-03-13T21:00:00Z.
```

```
Next update cadence: 3m.
```